PROGRAMMERING · JAVASCRIPT

Hent, regn, vis

JavaScript-hefte 1: variabler, knapper, input og utregninger

Eksempel · Oppgave A · Oppgave B · Huskeliste

Navn: _____

Klasse: _____

Slik bruker du heftet

I dette heftet samler du alt du har lært om JavaScript så langt: variabler, knapper, funksjoner, input-felt og utregninger. Først bygger du ett komplett program steg for steg sammen med heftet. Etterpå løser du to oppgaver på egen hånd.

Før du begynner

1. Åpne mappen `JS` i Visual Studio Code.
2. Lag en ny fil for eksempelet og for hver oppgave. Filnavnene står i heftet.
3. Skriv av koden selv. Ikke kopier – du lærer mer av å skrive.
4. Lagre (Ctrl+S) og åpne filen i nettleseren med Live Server.
5. Test etter **hvert** steg. Da vet du nøyaktig hvor feilen er hvis noe slutter å virke.
6. Skriv svarene dine på svarlinjene.

Det viktigste å huske: Hent – Regn – Vis

Nesten alle programmene vi lager følger den samme oppskriften. Brukeren skriver noe og trykker på en knapp. Da kjører en funksjon som gjør tre ting:

```
1. HENT                 2. REGN                 3. VIS
Les det brukeren   ──►   Regn ut svaret   ──►   Skriv svaret
skrev i input-feltet     med + - * /             på siden

Number(inpTall.value)    let svar = tall * 2     txtUt.textContent = ...
```

Når du står fast, spør deg selv: *Hva skal jeg hente? Hva skal jeg regne ut? Hva skal jeg vise?*

Eksempel: Alderskalkulatoren

Du skal lage et program der brukeren skriver inn alderen sin og får vite omtrent hvor mange dager og timer gammel hen er. Lag filen `eksempel-alder.html`.

Slik skal det se ut når du er ferdig

```
+------------------------------------------------+
|  Alderskalkulator                              |
|                                                |
|  Hvor gammel er du?                            |
|  [ 14         ]  [Regn ut]                     |
|                                                |
|  Du er ca. 5110 dager gammel. Det er 122640    |
|  timer!                                        |
+------------------------------------------------+
```

Steg 1 – Skjelettet

Skriv `!` og trykk Tab. Da får du skjelettet til en HTML-side. Endre `<title>` og legg inn en overskrift i `<body>`:

```
<!DOCTYPE html>
<html lang="no">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Alderskalkulator</title>
</head>
<body>
  <h1>Alderskalkulator</h1>

</body>
</html>
```

Lagre og sjekk at overskriften vises i nettleseren.

Steg 2 – HTML-en: input, knapp og et tomt avsnitt

Skriv dette under `<h1>`:

```
  <p>Hvor gammel er du?</p>
  <input type="number" id="inpAlder">
  <button onclick="regnUt()">Regn ut</button>
  <p id="txtUt"></p>
```

Legg merke til tre ting:

- Input-feltet har `id="inpAlder"`. Det er **navnet** JavaScript bruker for å finne feltet. Vi starter id-en med `inp` så vi husker at det er et input-felt.
- Knappen har `onclick="regnUt()"`. Når noen trykker, kjører funksjonen `regnUt`. Den har vi ikke laget ennå.
- Avsnittet `<p id="txtUt">` er tomt. Det er her svaret skal stå.

Lagre og se på siden. Hva skjer når du trykker på knappen nå? Hvorfor?

Svar: _____

Steg 3 – En funksjon som bare tester knappen

Skriv en `<script>`-blokk rett over `</body>`:

```
<script>
  function regnUt() {
    txtUt.textContent = "Knappen virker!";
  }
</script>
```

Lagre, oppdater og trykk på knappen. Står det «Knappen virker!» på siden?

Det kan virke dumt å skrive noe vi skal slette etterpå. Men nå **vet** du at knappen, funksjonen og `txtUt` henger sammen. Hvis noe går galt senere, er feilen et annet sted.

> **Tips:** Navnet i `onclick="regnUt()"` og navnet etter `function` må være helt like – også store og små bokstaver. `regnut` og `regnUt` er to forskjellige navn for datamaskinen.

Steg 4 – HENT tallet fra input-feltet

Bytt ut linjen inni funksjonen, så funksjonen ser slik ut:

```
  function regnUt() {
    let alder = Number(inpAlder.value);
    txtUt.textContent = `Du skrev ${alder}`;
  }
```

- `inpAlder.value` er det brukeren har skrevet i feltet.
- `Number( ... )` gjør det om til et tall.
- `let alder = ...` lagrer tallet i en variabel som heter `alder`.

Skriv inn alderen din, trykk på knappen og sjekk at tallet vises.

Undersøk: Endre den andre linjen til dette:

```
    txtUt.textContent = `Neste år blir du ${alder + 1}`;
```

Test med alderen din. Fjern så `Number(` og `)` fra den første linjen, slik at det står `let alder = inpAlder.value;`. Test igjen. Fyll ut tabellen.

Kode	Du skriver `14`. Hva står på siden?
`let alder = Number(inpAlder.value);`	
`let alder = inpAlder.value;`	

Hvorfor blir det forskjellig? Tenk på `"3" + "5"` fra den første JavaScript-timen.

Svar: _____

Sett `Number( ... )` tilbake før du går videre!

> **Husk:** Alt som kommer fra et input-felt er **tekst**, selv om det ser ut som et tall. Bruk alltid `Number( ... )` når du skal regne med det.

Steg 5 – REGN ut svaret

Et år har 365 dager, og et døgn har 24 timer. Legg til to nye linjer under `let alder`:

```
  let dager = alder * 365;
  let timer = dager * 24;
```

Vi bruker `*` for å gange. Legg merke til at `timer` bruker `dager`, som vi nettopp regnet ut. En variabel kan brukes i neste utregning.

Steg 6 – VIS svaret

Bytt ut linjen med `txtUt.textContent` med denne:

```
  txtUt.textContent = `Du er ca. ${dager} dager gammel. Det er ${timer} timer!`;
```

Teksten står mellom to **backticks** `` ` `` (ikke apostrofer `'`). Backtick lager du med Shift + tasten til venstre for Backspace, og så mellomrom. Inni backticks kan du sette inn verdien til en variabel med `${variabelnavn}`.

Hele funksjonen ser nå slik ut:

```
function regnUt() {
  let alder = Number(inpAlder.value);
  let dager = alder * 365;
  let timer = dager * 24;
  txtUt.textContent = `Du er ca. ${dager} dager gammel. Det er ${timer} timer!`;
}
```

Finn **Hent**, **Regn** og **Vis** i koden. Linjene inni funksjonen har nummer 1–4, fra `let alder` (1) til `txtUt.textContent` (4). Skriv linjenummeret for hver del.

Del	Linje(r) i funksjonen
Hent	
Regn	
Vis	

Undersøk: I «Hils på meg» brukte vi `document.write`. Bytt ut `txtUt.textContent = ...` med `document.write(...)` og trykk på knappen. Hva skjer med overskriften, input-feltet og knappen?

Svar: _____

Sett `txtUt.textContent` tilbake. Inni en funksjon bruker vi alltid `textContent`, fordi `document.write` sletter hele siden når den kjøres etter at siden er lastet.

Steg 7 – Hva om brukeren ikke skriver noe?

Trykk på knappen mens input-feltet er tomt. Hva står det?

Svar: _____

Et tomt felt blir `0` når vi bruker `Number`. Det kan vi sjekke med `if`, slik vi gjorde i nedtellingen. Endre funksjonen:

```
function regnUt() {
  let alder = Number(inpAlder.value);

  if (alder == 0) {
    txtUt.textContent = "Du må skrive inn alderen din først!";
  } else {
    let dager = alder * 365;
    let timer = dager * 24;
    txtUt.textContent = `Du er ca. ${dager} dager gammel. Det er ${timer} timer!`;
  }
}
```

- `if (alder == 0)` betyr «hvis alder er lik 0». Husk to likhetstegn når du **sjekker**. Ett likhetstegn betyr at du **gir** en variabel en verdi.
- Koden mellom de første `{ }` kjører bare hvis sjekken stemmer.
- Koden i `else { }` kjører hvis sjekken **ikke** stemmer.

Test både med tomt felt og med alderen din.

Steg 8 – Utvid og utforsk

1. Regn også ut **minutter** (`timer * 60`) og vis det i teksten.
2. Legg til et input-felt for navn: `<input type="text" id="inpNavn">`. Få svaret til å starte med navnet, for eksempel «Ola, du er ca. ...». Trenger du `Number( ... )` rundt navnet? Hvorfor / hvorfor ikke?
3. Skriv inn `-5`. Hva skjer? Endre `if`-sjekken til `if (alder <= 0)`. Hva betyr `<=`?
4. **Ekstra:** Legg til en knapp «Mørk modus» som gjør bakgrunnen mørk, slik vi gjorde i «Bytt bakgrunnsfarge».

Svar: _____

Svar: _____

Når koden ikke virker

Gå gjennom listen før du spør læreren:

- [ ] Er id-en **helt lik** i HTML-en og i JavaScript-en? (`inpAlder` er ikke det samme som `inpalder`.)
- [ ] Er navnet i `onclick` helt likt navnet på funksjonen, med `()` til slutt?
- [ ] Har alle `(` en `)`, og alle `{` en `}`?
- [ ] Brukte du backticks `` ` `` og ikke apostrofer `'` rundt teksten med `${ }`?
- [ ] Har du `Number( ... )` rundt `.value` når du skal regne?
- [ ] Trykk **F12** og velg fanen **Console**. Står det en rød feilmelding? Den forteller hvilken linje feilen er på.

Oppgave A: Kinokveld

Lag en kalkulator som regner ut hva en kinokveld koster. Brukeren skriver inn hvor mange billetter og hvor mange popcorn hen vil kjøpe. Nå får du ingen ferdig kode – men du kan se på eksempelet og på kalkulatoren vi laget i timen. Lag filen `oppgave-a-kino.html`.

Priser: En billett koster 130 kr. Et popcorn koster 70 kr.

Slik skal det se ut

```
+-------------------------------------------------+
|  Kinokveld                                      |
|                                                 |
|  Antall billetter:  [ 3      ]                  |
|  Antall popcorn:    [ 2      ]                  |
|  [Regn ut pris]                                 |
|                                                 |
|  Billetter: 390 kr. Popcorn: 140 kr.            |
|  Totalt: 530 kr.                                |
+-------------------------------------------------+
```

Krav – kryss av når du er ferdig

- [ ] Siden har en overskrift, to input-felt med `type="number"` og en knapp.
- [ ] Hvert input-felt har en id som starter med `inp`.
- [ ] Når du trykker på knappen, kjører en funksjon.
- [ ] Funksjonen henter begge tallene med `Number( ... )`.
- [ ] Funksjonen regner ut prisen for billettene, prisen for popcornet og totalprisen.
- [ ] Svaret vises i et avsnitt med `textContent` og `${ }`.

Hjelp til å komme i gang

- Planlegg før du koder. Fyll ut:
  - Hent: _____
  - Regn: _____
  - Vis: _____
- Lagre prisene i variabler, for eksempel `let billettpris = 130;`. Da er det lett å endre prisen senere.
- Prisen for billettene er antall billetter **ganger** billettprisen.
- Test etter hvert steg, akkurat som i eksempelet. Start med en funksjon som bare skriver «Knappen virker!».

Sjekk at svaret er riktig. Regn ut for hånd først, og sammenlign med programmet ditt.

Billetter	Popcorn	Totalt (regnet for hånd)	Totalt (fra programmet)
1	0		
2	2		
5	3		

Ekstra utfordring

- Legg til et felt for **antall personer**, og vis hva det koster per person. Hvilket regnetegn trenger du?
- Hvis antall billetter er 0, skal det stå «Du må kjøpe minst én billett!» i stedet for prisen.
- Lag en knapp «Nullstill» som tømmer feltene og svaret. Hint: `inpBilletter.value = "";`

Oppgave B: Sparegrisen

Lag en digital sparegris. Brukeren skriver inn et beløp og kan legge pengene **i** grisen eller ta dem **ut**. Grisen skal huske hvor mye som er i den – slik telleren husket antall trykk. Lag filen `oppgave-b-sparegris.html`.

Slik skal det se ut

```
+------------------------------------------------+
|  Sparegrisen                                   |
|                                                |
|  Beløp: [ 50       ]                           |
|  [Legg i]   [Ta ut]                            |
|                                                |
|  Du har 350 kr i sparegrisen.                  |
+------------------------------------------------+
```

Krav – kryss av når du er ferdig

- [ ] Det finnes en variabel `saldo` som starter på `0`. Den står **utenfor** funksjonene, slik `antallTrykk` gjorde i telleren.
- [ ] Knappen «Legg i» kjører en funksjon som legger beløpet til saldoen.
- [ ] Knappen «Ta ut» kjører en annen funksjon som trekker beløpet fra saldoen.
- [ ] Etter hvert trykk står det «Du har ... kr i sparegrisen.»
- [ ] Du kan ikke ta ut mer enn det som er i sparegrisen. Da skal det stå «Du har ikke nok penger!», og saldoen skal ikke endres.

Hjelp til å komme i gang

- Du trenger **to** funksjoner, for eksempel `leggI()` og `taUt()`. Begge starter med å hente beløpet.
- Slik legger du til et beløp i saldoen: `saldo = saldo + belop;` Les det som: «den nye saldoen er den gamle saldoen pluss beløpet».
- `>` betyr «større enn». Sjekken `if (belop > saldo)` stemmer når du prøver å ta ut mer enn du har.
- Unngå æ, ø og å i variabelnavn. Skriv `belop`, ikke `beløp`.
- Lag og test «Legg i» ferdig før du begynner på «Ta ut».

Test sparegrisen. Gjør handlingene i rekkefølge, og skriv hva som står på siden etter hver.

Handling	Hva står på siden?
Legg i 100	
Legg i 250	
Ta ut 50	
Ta ut 500	
Ta ut 300	

Ekstra utfordring

- Lag en knapp «Knus sparegrisen» som setter saldoen til 0.
- Tell hvor mange ganger det er lagt penger i grisen med `++`, og vis det.
- Gjør bakgrunnen grønn når saldoen er 1000 kr eller mer. Hint: `>=` betyr «større enn eller lik», og `document.body.style` kjenner du fra «Bytt bakgrunnsfarge».

Huskeliste

Kode	Hva den gjør
`let tall = 5;`	Lager en variabel og gir den en verdi.
`tall = tall + 2;`	Gir en variabel en ny verdi.
`+ - * /`	Pluss, minus, gange og dele.
`"3" + "5"`	Blir `"35"`. Tekst + tekst setter tekstene sammen.
`teller++;` / `teller--;`	Øker / minker en variabel med 1.
`function regnUt() { ... }`	Lager en funksjon. Koden inni kjører når funksjonen blir kalt.
`<button onclick="regnUt()">`	Kaller funksjonen når noen trykker på knappen.
`inpAlder.value`	Det brukeren har skrevet i input-feltet med `id="inpAlder"`. Alltid tekst!
`Number(inpAlder.value)`	Gjør teksten om til et tall, så du kan regne med det.
`txtUt.textContent = "Hei";`	Skriver tekst i elementet med `id="txtUt"`.
`` `Du er ${alder} år` ``	Tekst med backticks. `${ }` setter inn verdien til en variabel.
`if ( ... ) { } else { }`	Kjører den første delen hvis sjekken stemmer, ellers `else`-delen.
`== > < >= <=`	Lik, større enn, mindre enn, større eller lik, mindre eller lik.

Husk: Hent – Regn – Vis. Og test etter hvert steg!